

Figure 2: HTML form input for SQL injection to bypass user's name validation

id: 115 -	Name: parker 1
id: 125 -	Name: richard 0
id: 130 -	Name: clive 1
id: 135 -	Name: parker 0

Figure 3: Display of all records from the user's table

Figure 4: HTML form input for SQL injection to perform unrestricted query

id: 115 -	Name: parker 1
id: 135 -	Name: parker 0

Figure 5: Records having display restriction using *status = 1*

Prevention: Both the above-mentioned injections can be prevented by restricting the input field length to a given suitable number and allowing an alphanumeric field value, as shown in the following code snippet:

```
if (preg_match("/^\w{6,20}$/", $_POST['username']))
{
    $sql = "SELECT * FROM users WHERE username='".$_.$uname."'";
    echo $sql;
    $result = mysqli_query($conn,$sql);
    if ($result->num_rows > 0) {
        // output data of each row
        while($row = $result->fetch_assoc()) {
            echo "<br>id: " . $row["userid"]. " - Name: " .
            $row["username"]. " " . $row["status"]. "<br>";
        }
    }
    else {
        echo "0 results <br>";
    }
}
else {
    echo "<br>username not accepted<br>";
}
```

The regular expression pattern matching function `preg_match()` will filter out any non-alphanumeric character within the input string and will allow a string of length 6 to 20 characters only. For an input field value `'parker'`, the above filter and the subsequent query string will generate a query string as:

```
SELECT * FROM users WHERE username='parker'
```

...and will produce the following browsing list:

```
id: 115 - Name: parker 1
```

```
id: 135 - Name: parker 0
```

Multiple query restriction

One dangerous SQL injection menace is the multiple query statement in single query execution. This injection is harmful in the sense that one can inject a destructive data manipulation query along with a simple SELECT query statement. One can add multiple SQL statements with a SELECT statement as follows:

```
"parker'; DELETE FROM users;";
```

A multiple query statement can be injected into a Web based PHP routine through an HTML form as used earlier. The form can read multiple query statements and pass them into the server-side routines. Here is an example of multiple query injection by a Web form and its form routine. For clarity, the entire code is given. Those who are interested can experiment with this code for further study.

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "customer";
// Create a connection
$conn = new mysqli($servername, $username,
$password,$database);
// Check the connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
else{
    echo "Connected successfully<br>";
}
//*****Main Body *****
$username = $_POST['username'];
$sql = "SELECT * FROM users WHERE username='".$_.$uname;

if ($conn -> multi_query($sql)) {
    do {
        // Store first result set
        if ($result = $conn -> store_result()) {
            while ($row = $result -> fetch_row()) {
                //Display only first two fields
                printf("%s,%s\n", $row[0],$row[1]);echo "<br>";
            }
            $result -> free_result();
        }
    }
    // Mark the end of each record set
```